

Experiment 1: Implement MLP and CNN for handwritten digit classification. Compare the results in terms of accuracy, F1 score. Plot train vs validation curve. Report the test accuracy and F1 score

Use MNIST dataset. Split into train, val and test sets in the ratio 60: 20:20. Before split shuffle the data.

Tune hyperparameters : learning rate, epoch, batch size.

```
In [85]: import numpy as np
from tensorflow.keras.datasets import mnist
from sklearn.model_selection import train_test_split

# Load MNIST
(X_train_full, Y_train_full), (X_test_full, Y_test_full) = mnist.load_data()

# Combine the original train and test sets
X_data = np.concatenate((X_train_full, X_test_full), axis=0)
Y_data = np.concatenate((Y_train_full, Y_test_full), axis=0)

# First split: 60% train, 40% temporary
X_train, X_temp, Y_train, Y_temp = train_test_split(
    X_data,
    Y_data,
    test_size=0.4,
    random_state=42,
    shuffle=True
)

# Second split: divide the remaining 40% equally
X_val, X_test, Y_val, Y_test = train_test_split(
    X_temp,
    Y_temp,
    test_size=0.5,
    random_state=45,
    shuffle=True
)
```

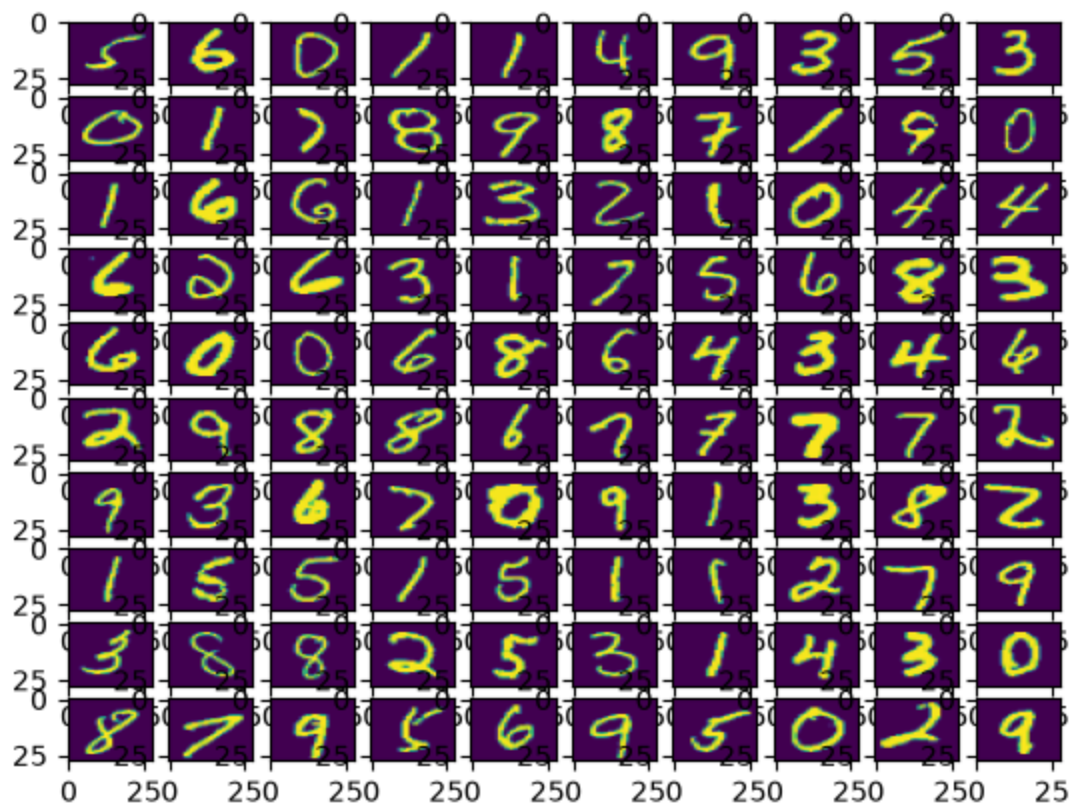
```
In [86]: # Check sizes
print("Train:", X_train.shape, Y_train.shape)
print("Validation:", X_val.shape, Y_val.shape)
print("Test:", X_test.shape, Y_test.shape)
```

```
Train: (42000, 28, 28) (42000,)
Validation: (14000, 28, 28) (14000,)
Test: (14000, 28, 28) (14000,)
```

```
In [87]: x_train_norm = X_train.astype("float32")/255
x_test_norm = X_test.astype("float32")/255
x_val_norm = X_val.astype("float32")/255
```

```
In [88]: import matplotlib.pyplot as plt
```

```
In [73]: fig, ax = plt.subplots(10, 10)
k = 0
for i in range(10):
    for j in range(10):
        ax[i][j].imshow(x_train_norm[k].reshape(28, 28), aspect='auto')
        k += 1
plt.show()
```



```
In [89]: import torch
import torch.nn as nn

print(torch.__version__)
print(torch.cuda.is_available())
```

2.13.0+cu130

False

```
In [47]: model = nn.Sequential(
    nn.Linear(784, 128),
    nn.ReLU(),
    nn.Linear(128, 64),
    nn.ReLU(),
    nn.Linear(64, 10)
)

print(model)
```

```
Sequential(
  (0): Linear(in_features=784, out_features=128, bias=True)
  (1): ReLU()
  (2): Linear(in_features=128, out_features=64, bias=True)
  (3): ReLU()
  (4): Linear(in_features=64, out_features=10, bias=True)
)
```

```
In [75]: device = torch.device(
          "cuda" if torch.cuda.is_available() else "cpu"
        )

        print("Device:", device)

        model = model.to(device)
```

Device: cpu

```
In [49]: # Convert NumPy arrays to PyTorch tensors

X_train_tensor = torch.tensor(x_train_norm, dtype=torch.float32)
X_val_tensor   = torch.tensor(x_val_norm, dtype=torch.float32)
X_test_tensor  = torch.tensor(x_test_norm, dtype=torch.float32)

Y_train_tensor = torch.tensor(Y_train, dtype=torch.long)
Y_val_tensor   = torch.tensor(Y_val, dtype=torch.long)
Y_test_tensor  = torch.tensor(Y_test, dtype=torch.long)
```

```
In [50]: X_train_tensor = X_train_tensor.reshape(-1, 784)
X_val_tensor   = X_val_tensor.reshape(-1, 784)
X_test_tensor  = X_test_tensor.reshape(-1, 784)
```

```
In [51]: from torch.utils.data import TensorDataset, DataLoader

train_dataset = TensorDataset(X_train_tensor, Y_train_tensor)
val_dataset   = TensorDataset(X_val_tensor, Y_val_tensor)
test_dataset  = TensorDataset(X_test_tensor, Y_test_tensor)
```

```
In [52]: batch_size = 10

train_loader = DataLoader(
    train_dataset,
    batch_size=batch_size,
    shuffle=True
)

val_loader = DataLoader(
    val_dataset,
    batch_size=batch_size,
    shuffle=False
)

test_loader = DataLoader(
    test_dataset,
    batch_size=batch_size,
```

```
    shuffle=False  
)
```

```
In [53]: criterion = nn.CrossEntropyLoss()
```

```
In [54]: learning_rate = 0.0001  
  
optimizer = torch.optim.Adam(  
    model.parameters(),  
    lr=learning_rate  
)
```

```
In [55]: epochs = 50  
  
train_losses = []  
val_losses = []  
  
train_accuracies = []  
val_accuracies = []  
  
for epoch in range(epochs):  
  
    # -----  
    # Training  
    # -----  
    model.train()  
  
    running_loss = 0.0  
    correct = 0  
    total = 0  
  
    for X_batch, Y_batch in train_loader:  
  
        X_batch = X_batch.to(device)  
        Y_batch = Y_batch.to(device)  
  
        # Forward pass  
        outputs = model(X_batch)  
  
        # Calculate loss  
        loss = criterion(outputs, Y_batch)  
  
        # Clear old gradients  
        optimizer.zero_grad()  
  
        # Backpropagation  
        loss.backward()  
  
        # Update weights  
        optimizer.step()  
  
        running_loss += loss.item()  
  
        # Predictions  
        _, predicted = torch.max(outputs, 1)
```

```

        total += Y_batch.size(0)
        correct += (predicted == Y_batch).sum().item()

train_loss = running_loss / len(train_loader)
train_accuracy = correct / total

# -----
# Validation
# -----
model.eval()

running_val_loss = 0.0
correct = 0
total = 0

with torch.no_grad():

    for X_batch, Y_batch in val_loader:

        X_batch = X_batch.to(device)
        Y_batch = Y_batch.to(device)

        outputs = model(X_batch)

        loss = criterion(outputs, Y_batch)

        running_val_loss += loss.item()

        _, predicted = torch.max(outputs, 1)

        total += Y_batch.size(0)
        correct += (predicted == Y_batch).sum().item()

val_loss = running_val_loss / len(val_loader)
val_accuracy = correct / total

# Store results
train_losses.append(train_loss)
val_losses.append(val_loss)

train_accuracies.append(train_accuracy)
val_accuracies.append(val_accuracy)

print(
    f"Epoch [{epoch+1}/{epochs}] "
    f"Train Loss: {train_loss:.4f} "
    f"Train Acc: {train_accuracy:.4f} "
    f"Val Loss: {val_loss:.4f} "
    f"Val Acc: {val_accuracy:.4f}"
)

```

Epoch [1/50] Train Loss: 0.5812 Train Acc: 0.8523 Val Loss: 0.3118 Val Acc: 0.9124
Epoch [2/50] Train Loss: 0.2748 Train Acc: 0.9213 Val Loss: 0.2491 Val Acc: 0.9306
Epoch [3/50] Train Loss: 0.2240 Train Acc: 0.9358 Val Loss: 0.2164 Val Acc: 0.9403
Epoch [4/50] Train Loss: 0.1874 Train Acc: 0.9457 Val Loss: 0.1910 Val Acc: 0.9432
Epoch [5/50] Train Loss: 0.1612 Train Acc: 0.9528 Val Loss: 0.1679 Val Acc: 0.9528
Epoch [6/50] Train Loss: 0.1401 Train Acc: 0.9588 Val Loss: 0.1516 Val Acc: 0.9572
Epoch [7/50] Train Loss: 0.1224 Train Acc: 0.9643 Val Loss: 0.1423 Val Acc: 0.9585
Epoch [8/50] Train Loss: 0.1083 Train Acc: 0.9687 Val Loss: 0.1313 Val Acc: 0.9619
Epoch [9/50] Train Loss: 0.0960 Train Acc: 0.9722 Val Loss: 0.1252 Val Acc: 0.9637
Epoch [10/50] Train Loss: 0.0864 Train Acc: 0.9747 Val Loss: 0.1174 Val Acc: 0.9654
Epoch [11/50] Train Loss: 0.0774 Train Acc: 0.9780 Val Loss: 0.1152 Val Acc: 0.9659
Epoch [12/50] Train Loss: 0.0702 Train Acc: 0.9796 Val Loss: 0.1076 Val Acc: 0.9679
Epoch [13/50] Train Loss: 0.0632 Train Acc: 0.9817 Val Loss: 0.1072 Val Acc: 0.9685
Epoch [14/50] Train Loss: 0.0570 Train Acc: 0.9834 Val Loss: 0.1022 Val Acc: 0.9702
Epoch [15/50] Train Loss: 0.0516 Train Acc: 0.9857 Val Loss: 0.1017 Val Acc: 0.9701
Epoch [16/50] Train Loss: 0.0466 Train Acc: 0.9869 Val Loss: 0.1017 Val Acc: 0.9681
Epoch [17/50] Train Loss: 0.0418 Train Acc: 0.9885 Val Loss: 0.1024 Val Acc: 0.9698
Epoch [18/50] Train Loss: 0.0381 Train Acc: 0.9896 Val Loss: 0.0984 Val Acc: 0.9716
Epoch [19/50] Train Loss: 0.0342 Train Acc: 0.9909 Val Loss: 0.1031 Val Acc: 0.9693
Epoch [20/50] Train Loss: 0.0306 Train Acc: 0.9920 Val Loss: 0.1013 Val Acc: 0.9701
Epoch [21/50] Train Loss: 0.0269 Train Acc: 0.9931 Val Loss: 0.1001 Val Acc: 0.9715
Epoch [22/50] Train Loss: 0.0245 Train Acc: 0.9935 Val Loss: 0.1035 Val Acc: 0.9683
Epoch [23/50] Train Loss: 0.0215 Train Acc: 0.9948 Val Loss: 0.1014 Val Acc: 0.9714
Epoch [24/50] Train Loss: 0.0194 Train Acc: 0.9956 Val Loss: 0.1017 Val Acc: 0.9719
Epoch [25/50] Train Loss: 0.0172 Train Acc: 0.9959 Val Loss: 0.0979 Val Acc: 0.9730
Epoch [26/50] Train Loss: 0.0154 Train Acc: 0.9967 Val Loss: 0.1009 Val Acc: 0.9721
Epoch [27/50] Train Loss: 0.0135 Train Acc: 0.9971 Val Loss: 0.1048 Val Acc: 0.9731
Epoch [28/50] Train Loss: 0.0118 Train Acc: 0.9974 Val Loss: 0.1109 Val Acc: 0.9709

```
Epoch [29/50] Train Loss: 0.0102 Train Acc: 0.9982 Val Loss: 0.1038 Val Acc: 0.9726
Epoch [30/50] Train Loss: 0.0092 Train Acc: 0.9983 Val Loss: 0.1079 Val Acc: 0.9724
Epoch [31/50] Train Loss: 0.0083 Train Acc: 0.9985 Val Loss: 0.1081 Val Acc: 0.9734
Epoch [32/50] Train Loss: 0.0069 Train Acc: 0.9989 Val Loss: 0.1121 Val Acc: 0.9728
Epoch [33/50] Train Loss: 0.0062 Train Acc: 0.9992 Val Loss: 0.1162 Val Acc: 0.9717
Epoch [34/50] Train Loss: 0.0056 Train Acc: 0.9991 Val Loss: 0.1159 Val Acc: 0.9729
Epoch [35/50] Train Loss: 0.0046 Train Acc: 0.9994 Val Loss: 0.1239 Val Acc: 0.9712
Epoch [36/50] Train Loss: 0.0041 Train Acc: 0.9995 Val Loss: 0.1186 Val Acc: 0.9728
Epoch [37/50] Train Loss: 0.0035 Train Acc: 0.9997 Val Loss: 0.1235 Val Acc: 0.9739
Epoch [38/50] Train Loss: 0.0030 Train Acc: 0.9998 Val Loss: 0.1227 Val Acc: 0.9737
Epoch [39/50] Train Loss: 0.0027 Train Acc: 0.9998 Val Loss: 0.1271 Val Acc: 0.9726
Epoch [40/50] Train Loss: 0.0027 Train Acc: 0.9996 Val Loss: 0.1284 Val Acc: 0.9727
Epoch [41/50] Train Loss: 0.0021 Train Acc: 0.9998 Val Loss: 0.1285 Val Acc: 0.9735
Epoch [42/50] Train Loss: 0.0021 Train Acc: 0.9997 Val Loss: 0.1300 Val Acc: 0.9741
Epoch [43/50] Train Loss: 0.0017 Train Acc: 0.9998 Val Loss: 0.1335 Val Acc: 0.9733
Epoch [44/50] Train Loss: 0.0015 Train Acc: 0.9999 Val Loss: 0.1337 Val Acc: 0.9739
Epoch [45/50] Train Loss: 0.0016 Train Acc: 0.9997 Val Loss: 0.1362 Val Acc: 0.9732
Epoch [46/50] Train Loss: 0.0012 Train Acc: 0.9999 Val Loss: 0.1389 Val Acc: 0.9736
Epoch [47/50] Train Loss: 0.0012 Train Acc: 0.9998 Val Loss: 0.1393 Val Acc: 0.9731
Epoch [48/50] Train Loss: 0.0015 Train Acc: 0.9997 Val Loss: 0.1457 Val Acc: 0.9722
Epoch [49/50] Train Loss: 0.0007 Train Acc: 1.0000 Val Loss: 0.1440 Val Acc: 0.9728
Epoch [50/50] Train Loss: 0.0008 Train Acc: 0.9999 Val Loss: 0.1466 Val Acc: 0.9732
```

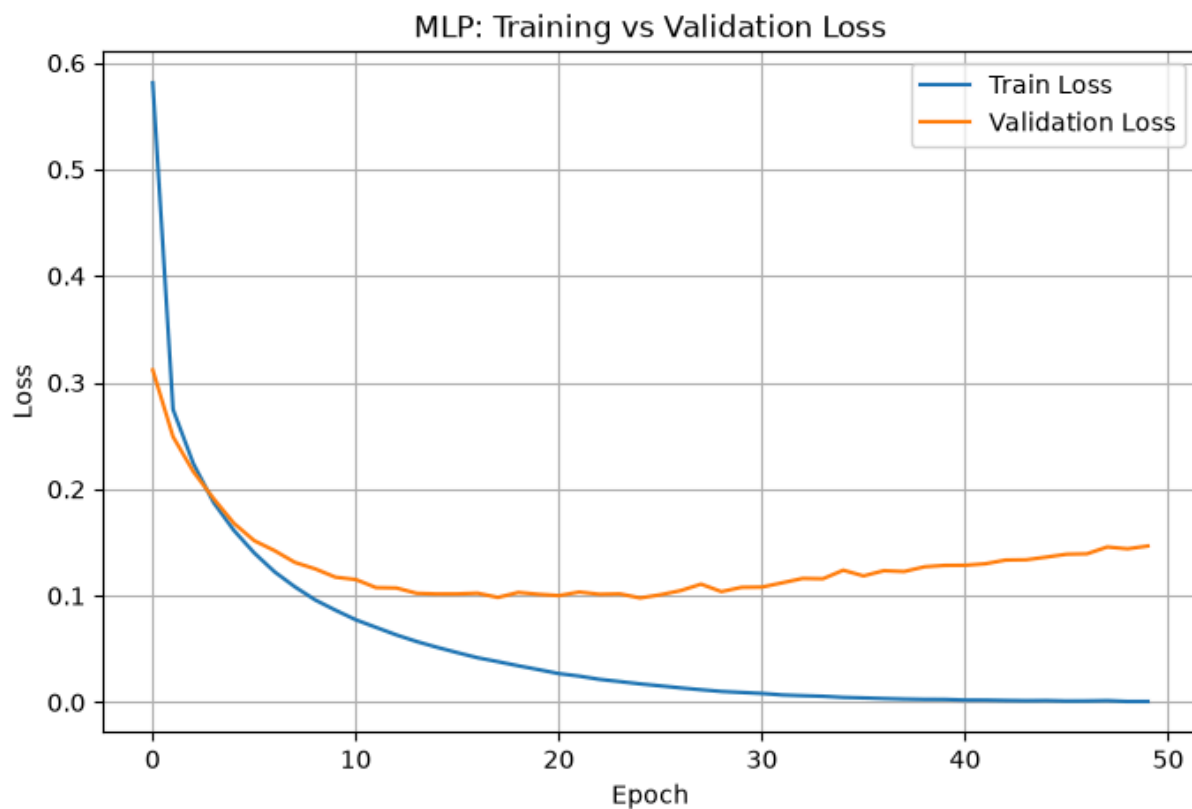
```
In [56]: import matplotlib.pyplot as plt

plt.figure(figsize=(8, 5))

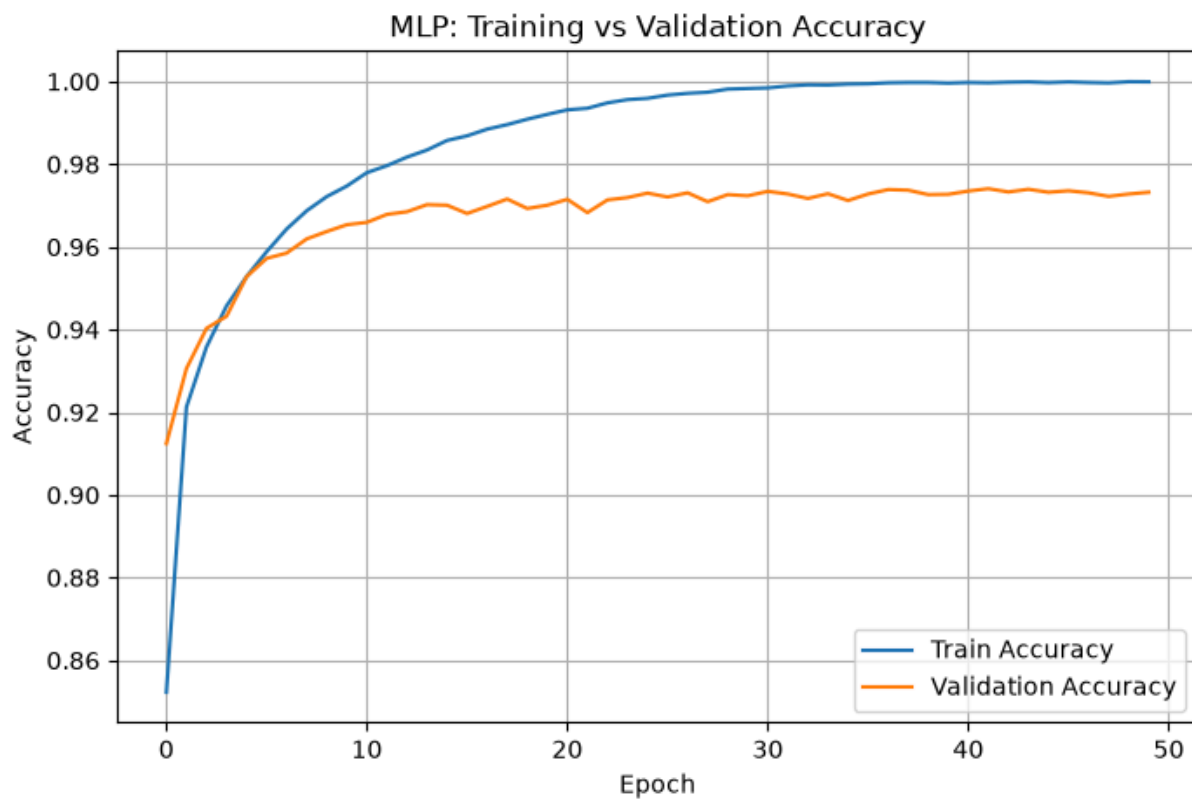
plt.plot(train_losses, label="Train Loss")
plt.plot(val_losses, label="Validation Loss")

plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("MLP: Training vs Validation Loss")
```

```
plt.legend()  
plt.grid()  
plt.show()
```



```
In [57]: plt.figure(figsize=(8, 5))  
  
plt.plot(train_accuracies, label="Train Accuracy")  
plt.plot(val_accuracies, label="Validation Accuracy")  
  
plt.xlabel("Epoch")  
plt.ylabel("Accuracy")  
plt.title("MLP: Training vs Validation Accuracy")  
  
plt.legend()  
plt.grid()  
plt.show()
```

```
In [58]: from sklearn.metrics import accuracy_score, f1_score

model.eval()

all_predictions = []
all_labels = []

with torch.no_grad():
    for X_batch, Y_batch in test_loader:

        X_batch = X_batch.to(device)
        Y_batch = Y_batch.to(device)

        outputs = model(X_batch)

        _, predictions = torch.max(outputs, 1)

        all_predictions.extend(predictions.cpu().numpy())
        all_labels.extend(Y_batch.cpu().numpy())
```

```
In [59]: test_accuracy = accuracy_score(
    all_labels,
    all_predictions
)

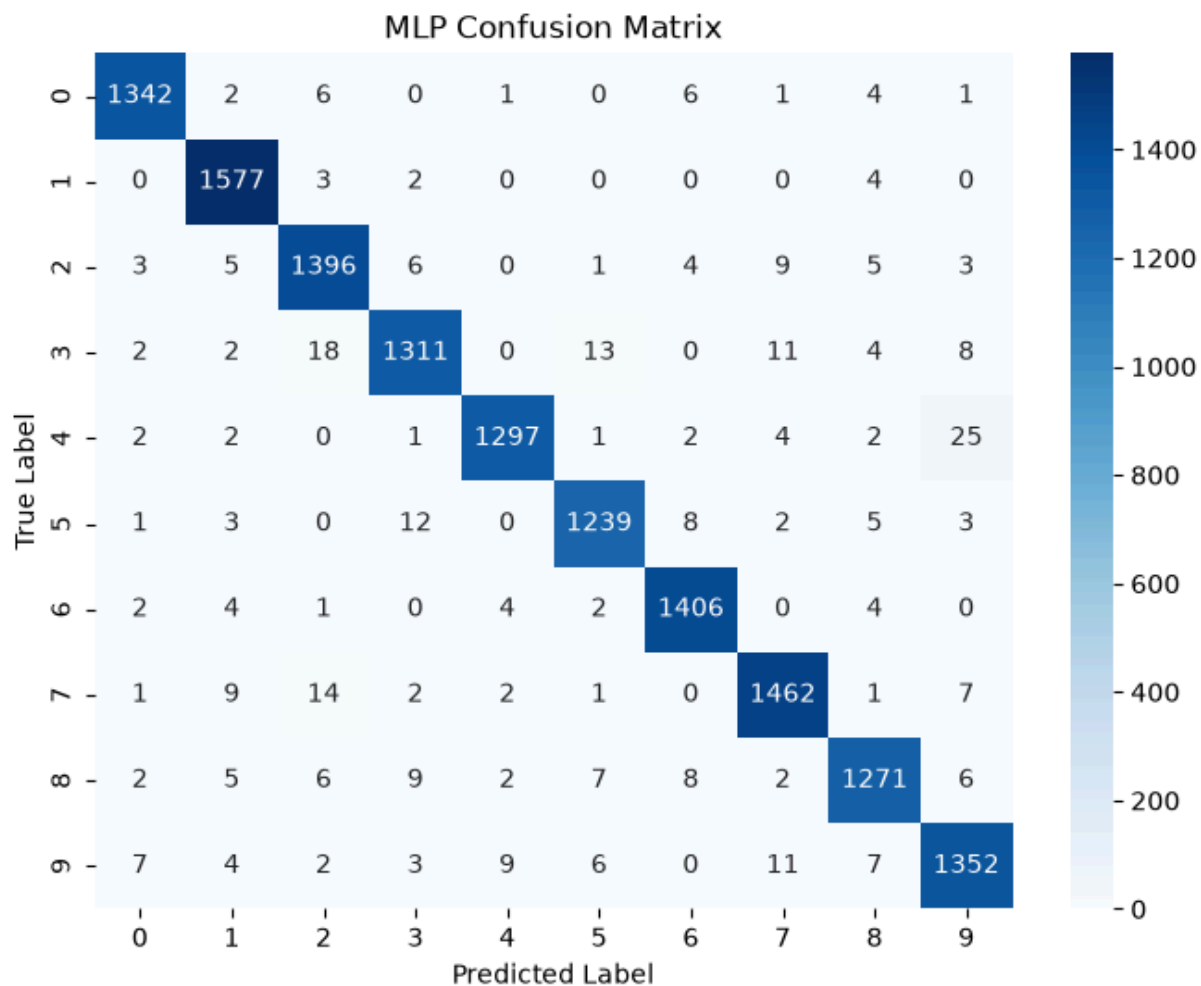
test_f1 = f1_score(
    all_labels,
    all_predictions,
    average="weighted"
```

```
)  
  
print(f"MLP Test Accuracy: {test_accuracy:.4f}")  
print(f"MLP Test F1 Score: {test_f1:.4f}")
```

MLP Test Accuracy: 0.9752

MLP Test F1 Score: 0.9752

```
In [60]: from sklearn.metrics import confusion_matrix  
import seaborn as sns  
import matplotlib.pyplot as plt  
  
cm = confusion_matrix(all_labels, all_predictions)  
  
plt.figure(figsize=(8, 6))  
  
sns.heatmap(  
    cm,  
    annot=True,  
    fmt="d",  
    cmap="Blues",  
    xticklabels=range(10),  
    yticklabels=range(10)  
)  
  
plt.xlabel("Predicted Label")  
plt.ylabel("True Label")  
plt.title("MLP Confusion Matrix")  
  
plt.show()
```



```
In [90]: model = nn.Sequential(
    nn.Conv2d(1, 32, kernel_size=3, padding=1),
    nn.ReLU(),
    nn.MaxPool2d(2),

    nn.Conv2d(32, 64, kernel_size=3, padding=1),
    nn.ReLU(),
    nn.MaxPool2d(2),

    nn.Flatten(),

    nn.Linear(64 * 7 * 7, 128),
    nn.ReLU(),

    nn.Linear(128, 10)
)
```

```
In [92]: # Convert NumPy arrays to PyTorch tensors

X_train_tensor = torch.tensor(x_train_norm, dtype=torch.float32)
X_val_tensor   = torch.tensor(x_val_norm, dtype=torch.float32)
X_test_tensor  = torch.tensor(x_test_norm, dtype=torch.float32)

Y_train_tensor = torch.tensor(Y_train, dtype=torch.long)
```

```
Y_val_tensor = torch.tensor(Y_val, dtype=torch.long)
Y_test_tensor = torch.tensor(Y_test, dtype=torch.long)
```

In [93]: *# Add channel dimension for CNN*

```
X_train_tensor = X_train_tensor.unsqueeze(1)
X_val_tensor = X_val_tensor.unsqueeze(1)
X_test_tensor = X_test_tensor.unsqueeze(1)
```

In [94]:

```
print(X_train_tensor.shape)
print(X_val_tensor.shape)
print(X_test_tensor.shape)
```

```
torch.Size([42000, 1, 28, 28])
torch.Size([14000, 1, 28, 28])
torch.Size([14000, 1, 28, 28])
```

In [95]: **from** torch.utils.data **import** TensorDataset, DataLoader

```
train_dataset = TensorDataset(X_train_tensor, Y_train_tensor)
val_dataset = TensorDataset(X_val_tensor, Y_val_tensor)
test_dataset = TensorDataset(X_test_tensor, Y_test_tensor)
```

In [96]: **batch_size** = 10

```
train_loader = DataLoader(
    train_dataset,
    batch_size=batch_size,
    shuffle=True
)
```

```
val_loader = DataLoader(
    val_dataset,
    batch_size=batch_size,
    shuffle=False
)
```

```
test_loader = DataLoader(
    test_dataset,
    batch_size=batch_size,
    shuffle=False
)
```

In [97]: **criterion** = nn.CrossEntropyLoss()

In [98]: **learning_rate** = 0.0001

```
optimizer = torch.optim.Adam(
    model.parameters(),
    lr=learning_rate
)
```

In [99]: **epochs** = 50

```
train_losses = []
```

```
val_losses = []

train_accuracies = []
val_accuracies = []

for epoch in range(epochs):

    # -----
    # Training
    # -----
    model.train()

    running_loss = 0.0
    correct = 0
    total = 0

    for X_batch, Y_batch in train_loader:

        X_batch = X_batch.to(device)
        Y_batch = Y_batch.to(device)

        # Forward pass
        outputs = model(X_batch)

        # Calculate loss
        loss = criterion(outputs, Y_batch)

        # Clear old gradients
        optimizer.zero_grad()

        # Backpropagation
        loss.backward()

        # Update weights
        optimizer.step()

        running_loss += loss.item()

        # Predictions
        _, predicted = torch.max(outputs, 1)

        total += Y_batch.size(0)
        correct += (predicted == Y_batch).sum().item()

    train_loss = running_loss / len(train_loader)
    train_accuracy = correct / total

    # -----
    # Validation
    # -----
    model.eval()

    running_val_loss = 0.0
    correct = 0
    total = 0
```

```
with torch.no_grad():

    for X_batch, Y_batch in val_loader:

        X_batch = X_batch.to(device)
        Y_batch = Y_batch.to(device)

        outputs = model(X_batch)

        loss = criterion(outputs, Y_batch)

        running_val_loss += loss.item()

        _, predicted = torch.max(outputs, 1)

        total += Y_batch.size(0)
        correct += (predicted == Y_batch).sum().item()

val_loss = running_val_loss / len(val_loader)
val_accuracy = correct / total

# Store results
train_losses.append(train_loss)
val_losses.append(val_loss)

train_accuracies.append(train_accuracy)
val_accuracies.append(val_accuracy)

print(
    f"Epoch [{epoch+1}/{epochs}] "
    f"Train Loss: {train_loss:.4f} "
    f"Train Acc: {train_accuracy:.4f} "
    f"Val Loss: {val_loss:.4f} "
    f"Val Acc: {val_accuracy:.4f}"
)
```

Epoch [1/50] Train Loss: 0.3040 Train Acc: 0.9100 Val Loss: 0.1287 Val Acc: 0.9634
Epoch [2/50] Train Loss: 0.0882 Train Acc: 0.9742 Val Loss: 0.0699 Val Acc: 0.9787
Epoch [3/50] Train Loss: 0.0614 Train Acc: 0.9811 Val Loss: 0.0587 Val Acc: 0.9822
Epoch [4/50] Train Loss: 0.0476 Train Acc: 0.9855 Val Loss: 0.0552 Val Acc: 0.9827
Epoch [5/50] Train Loss: 0.0378 Train Acc: 0.9888 Val Loss: 0.0479 Val Acc: 0.9856
Epoch [6/50] Train Loss: 0.0320 Train Acc: 0.9897 Val Loss: 0.0394 Val Acc: 0.9878
Epoch [7/50] Train Loss: 0.0264 Train Acc: 0.9918 Val Loss: 0.0430 Val Acc: 0.9866
Epoch [8/50] Train Loss: 0.0217 Train Acc: 0.9932 Val Loss: 0.0449 Val Acc: 0.9865
Epoch [9/50] Train Loss: 0.0182 Train Acc: 0.9943 Val Loss: 0.0484 Val Acc: 0.9853
Epoch [10/50] Train Loss: 0.0151 Train Acc: 0.9953 Val Loss: 0.0382 Val Acc: 0.9882
Epoch [11/50] Train Loss: 0.0126 Train Acc: 0.9963 Val Loss: 0.0383 Val Acc: 0.9889
Epoch [12/50] Train Loss: 0.0112 Train Acc: 0.9965 Val Loss: 0.0399 Val Acc: 0.9886
Epoch [13/50] Train Loss: 0.0091 Train Acc: 0.9975 Val Loss: 0.0378 Val Acc: 0.9890
Epoch [14/50] Train Loss: 0.0077 Train Acc: 0.9979 Val Loss: 0.0420 Val Acc: 0.9891
Epoch [15/50] Train Loss: 0.0069 Train Acc: 0.9980 Val Loss: 0.0601 Val Acc: 0.9839
Epoch [16/50] Train Loss: 0.0059 Train Acc: 0.9981 Val Loss: 0.0455 Val Acc: 0.9889
Epoch [17/50] Train Loss: 0.0048 Train Acc: 0.9988 Val Loss: 0.0403 Val Acc: 0.9897
Epoch [18/50] Train Loss: 0.0046 Train Acc: 0.9987 Val Loss: 0.0631 Val Acc: 0.9861
Epoch [19/50] Train Loss: 0.0039 Train Acc: 0.9988 Val Loss: 0.0431 Val Acc: 0.9896
Epoch [20/50] Train Loss: 0.0034 Train Acc: 0.9988 Val Loss: 0.0483 Val Acc: 0.9891
Epoch [21/50] Train Loss: 0.0025 Train Acc: 0.9993 Val Loss: 0.0461 Val Acc: 0.9901
Epoch [22/50] Train Loss: 0.0030 Train Acc: 0.9991 Val Loss: 0.0484 Val Acc: 0.9894
Epoch [23/50] Train Loss: 0.0032 Train Acc: 0.9989 Val Loss: 0.0606 Val Acc: 0.9862
Epoch [24/50] Train Loss: 0.0018 Train Acc: 0.9995 Val Loss: 0.0449 Val Acc: 0.9904
Epoch [25/50] Train Loss: 0.0020 Train Acc: 0.9995 Val Loss: 0.0491 Val Acc: 0.9893
Epoch [26/50] Train Loss: 0.0017 Train Acc: 0.9996 Val Loss: 0.0553 Val Acc: 0.9883
Epoch [27/50] Train Loss: 0.0019 Train Acc: 0.9994 Val Loss: 0.0491 Val Acc: 0.9899
Epoch [28/50] Train Loss: 0.0017 Train Acc: 0.9995 Val Loss: 0.0634 Val Acc: 0.9874

```
Epoch [29/50] Train Loss: 0.0019 Train Acc: 0.9995 Val Loss: 0.0510 Val Acc: 0.9901
Epoch [30/50] Train Loss: 0.0011 Train Acc: 0.9996 Val Loss: 0.0639 Val Acc: 0.9879
Epoch [31/50] Train Loss: 0.0018 Train Acc: 0.9993 Val Loss: 0.0497 Val Acc: 0.9904
Epoch [32/50] Train Loss: 0.0006 Train Acc: 0.9998 Val Loss: 0.0512 Val Acc: 0.9905
Epoch [33/50] Train Loss: 0.0014 Train Acc: 0.9995 Val Loss: 0.0612 Val Acc: 0.9880
Epoch [34/50] Train Loss: 0.0009 Train Acc: 0.9998 Val Loss: 0.0588 Val Acc: 0.9901
Epoch [35/50] Train Loss: 0.0012 Train Acc: 0.9997 Val Loss: 0.0678 Val Acc: 0.9884
Epoch [36/50] Train Loss: 0.0012 Train Acc: 0.9996 Val Loss: 0.0592 Val Acc: 0.9898
Epoch [37/50] Train Loss: 0.0013 Train Acc: 0.9996 Val Loss: 0.0579 Val Acc: 0.9901
Epoch [38/50] Train Loss: 0.0003 Train Acc: 0.9999 Val Loss: 0.0750 Val Acc: 0.9876
Epoch [39/50] Train Loss: 0.0015 Train Acc: 0.9995 Val Loss: 0.0647 Val Acc: 0.9891
Epoch [40/50] Train Loss: 0.0008 Train Acc: 0.9996 Val Loss: 0.0573 Val Acc: 0.9899
Epoch [41/50] Train Loss: 0.0015 Train Acc: 0.9995 Val Loss: 0.0545 Val Acc: 0.9904
Epoch [42/50] Train Loss: 0.0005 Train Acc: 0.9999 Val Loss: 0.0589 Val Acc: 0.9899
Epoch [43/50] Train Loss: 0.0013 Train Acc: 0.9996 Val Loss: 0.0696 Val Acc: 0.9876
Epoch [44/50] Train Loss: 0.0003 Train Acc: 1.0000 Val Loss: 0.0583 Val Acc: 0.9905
Epoch [45/50] Train Loss: 0.0008 Train Acc: 0.9996 Val Loss: 0.0651 Val Acc: 0.9901
Epoch [46/50] Train Loss: 0.0008 Train Acc: 0.9998 Val Loss: 0.0676 Val Acc: 0.9894
Epoch [47/50] Train Loss: 0.0003 Train Acc: 0.9999 Val Loss: 0.0588 Val Acc: 0.9907
Epoch [48/50] Train Loss: 0.0010 Train Acc: 0.9997 Val Loss: 0.0669 Val Acc: 0.9892
Epoch [49/50] Train Loss: 0.0004 Train Acc: 0.9999 Val Loss: 0.0562 Val Acc: 0.9908
Epoch [50/50] Train Loss: 0.0005 Train Acc: 0.9999 Val Loss: 0.0756 Val Acc: 0.9891
```

```
In [100... import matplotlib.pyplot as plt

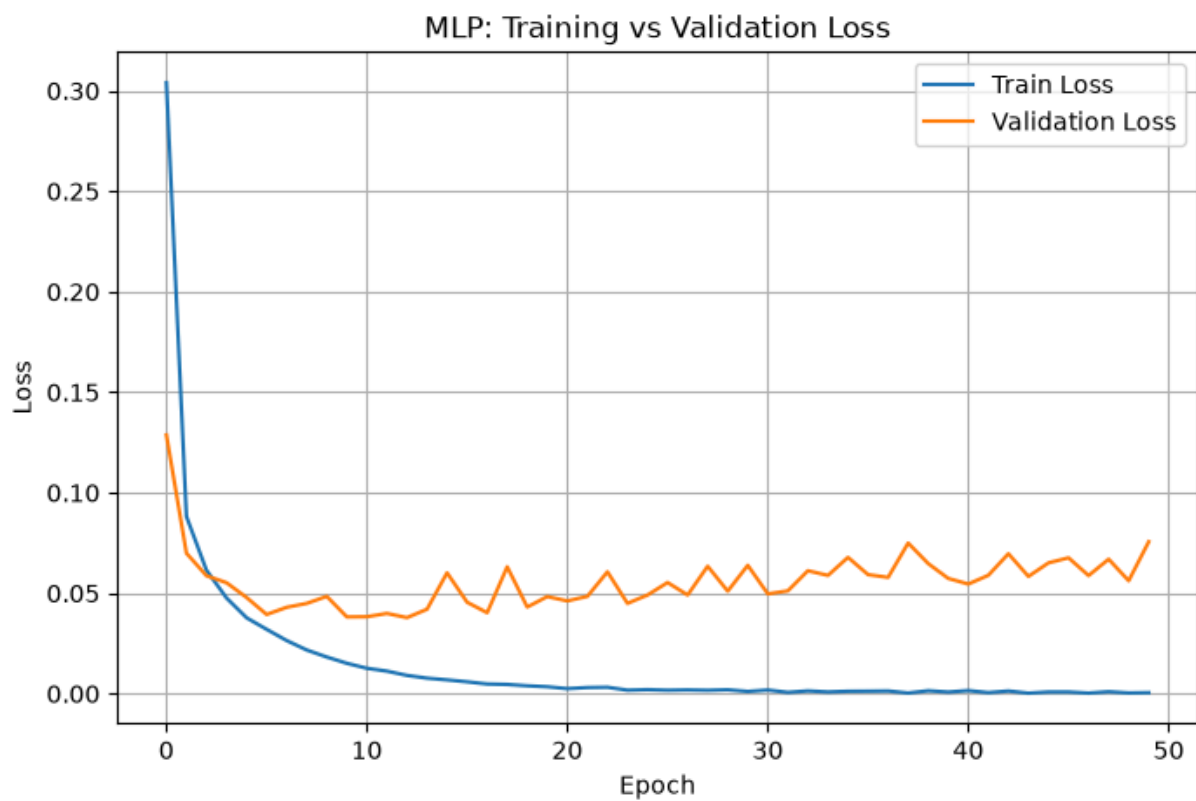
plt.figure(figsize=(8, 5))

plt.plot(train_losses, label="Train Loss")
plt.plot(val_losses, label="Validation Loss")

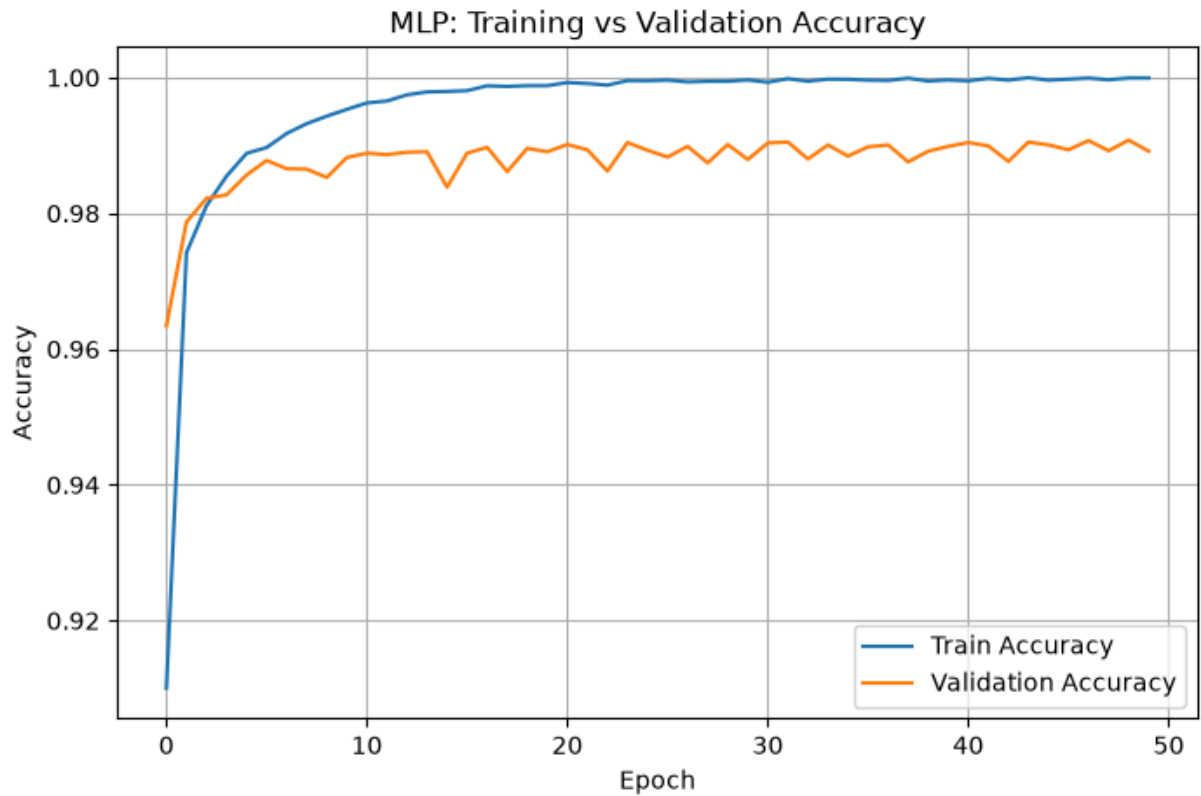
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("MLP: Training vs Validation Loss")
```



```
plt.legend()  
plt.grid()  
plt.show()
```



```
In [101... plt.figure(figsize=(8, 5))  
  
plt.plot(train_accuracies, label="Train Accuracy")  
plt.plot(val_accuracies, label="Validation Accuracy")  
  
plt.xlabel("Epoch")  
plt.ylabel("Accuracy")  
plt.title("MLP: Training vs Validation Accuracy")  
  
plt.legend()  
plt.grid()  
plt.show()
```



```
In [102... from sklearn.metrics import accuracy_score, f1_score

model.eval()

all_predictions = []
all_labels = []

with torch.no_grad():

    for X_batch, Y_batch in test_loader:

        X_batch = X_batch.to(device)
        Y_batch = Y_batch.to(device)

        outputs = model(X_batch)

        _, predictions = torch.max(outputs, 1)

        all_predictions.extend(predictions.cpu().numpy())
        all_labels.extend(Y_batch.cpu().numpy())
```

```
In [103... test_accuracy = accuracy_score(
    all_labels,
    all_predictions
)

test_f1 = f1_score(
    all_labels,
    all_predictions,
    average="weighted"
```

```
)  
  
print(f"MLP Test Accuracy: {test_accuracy:.4f}")  
print(f"MLP Test F1 Score: {test_f1:.4f}")
```

MLP Test Accuracy: 0.9892

MLP Test F1 Score: 0.9892

```
In [104... from sklearn.metrics import confusion_matrix  
import seaborn as sns  
import matplotlib.pyplot as plt  
  
cm = confusion_matrix(all_labels, all_predictions)  
  
plt.figure(figsize=(8, 6))  
  
sns.heatmap(  
    cm,  
    annot=True,  
    fmt="d",  
    cmap="Blues",  
    xticklabels=range(10),  
    yticklabels=range(10)  
)  
  
plt.xlabel("Predicted Label")  
plt.ylabel("True Label")  
plt.title("MLP Confusion Matrix")  
  
plt.show()
```

